

# Standard Operating Procedure (SOP)

**Title:** Flutter Application Development Using GetX

## Purpose:

The purpose of this SOP is to establish standardized development guidelines for Flutter applications using GetX, ensuring consistency, maintainability, scalability, and ease of onboarding across all Flutter projects.

## Scope:

This SOP applies to:

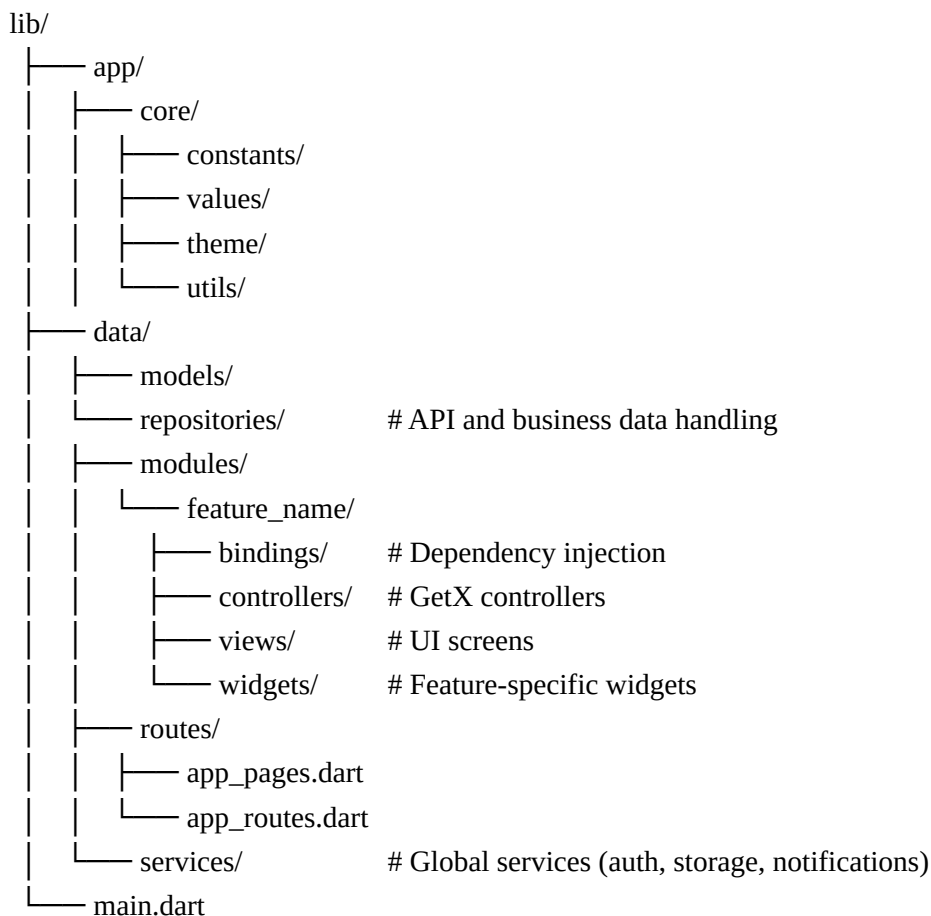
- All new Flutter projects
- Enhancements and refactoring of existing Flutter projects
- All Flutter developers, reviewers, and tech leads

## Technology Standards:

- Framework: Flutter (Stable channel only)
- Language: Dart (Null Safety mandatory)
- State Management: GetX (Mandatory)
- Architecture: Feature-based modular architecture
- Version Control: Git

## Standard Project Structure (GetX-Based)

All Flutter projects must strictly follow the GetX-recommended, feature-first modular structure defined below:



## Best Practices & Rules

- Feature-first approach is mandatory; avoid layer-based global UI folders
- Each feature must be fully self-contained inside its module
- controllers must contain only state and business logic
- views must contain UI code only (no API or business logic)
- API calls must be handled via repositories only
- Use widgets only for reusable UI components within the same feature
- Global reusable components must be placed under core
- Avoid circular dependencies between modules
- Folder and file names must use lowercase\_with\_underscores

## GetX Usage Guidelines

GetX must be used in a disciplined and consistent manner across all projects.

### Controller Best Practices

- Each screen or feature must have its own dedicated GetxController
- Controllers must be lightweight and focused on state and orchestration only
- Avoid placing heavy business logic directly in controllers; delegate to repositories
- Use onInit() for initial API calls and setup logic
- Use onClose() to dispose streams, timers, and controllers
- Do not instantiate controllers manually inside widgets; always use Bindings

### State Management Rules

- Use Rx variables only where reactivity is required
- Avoid excessive use of Rx for static or non-changing values
- Group related state variables logically
- Prefer Obx or GetX widgets over deeply nested reactive widgets

### Navigation & Dependency Injection

- All navigation must use GetX navigation methods
- Use Bindings to manage dependency lifecycle
- Avoid global permanent controllers unless explicitly required

## Best Practices (Mandatory)

### Architecture & Design

- Follow feature-first modular architecture strictly
- Ensure clear separation between UI, state, and data layers
- Avoid tight coupling between modules
- Prefer composition over inheritance

### UI Development

- Keep widgets small and reusable
- Avoid business logic inside widgets
- Use feature-level widgets before creating global widgets

- Maintain consistent spacing, typography, and theming

## Error Handling

- Handle API failures gracefully
- Display user-friendly error messages
- Log technical errors for debugging

## Performance

- Avoid unnecessary widget rebuilds
- Dispose unused controllers properly
- Optimize lists using ListView.builder

## Security

- Never hardcode API keys or secrets
- Avoid logging sensitive user data
- Secure local storage usage

# **Coding Standards**

## Mandatory Practices

- Follow Dart and Flutter naming conventions
- Widgets should not exceed 200 lines
- Create reusable widgets for repeated UI components
- Handle API errors and null values explicitly
- Avoid hardcoded strings, colors, and styles

## Linting

- Flutter lint rules must be enabled
- Code must compile without warnings before PR submission

# **Git Branching & Version Control**

## Branch Strategy

- main – Production
- develop – Development
- feature/feature-name
- bugfix/bug-name

## Rules

- No direct commits to main or develop
- All changes must go through Pull Requests
- PR review by Tech Lead or assigned reviewer is mandatory
- Commit messages must be clear and descriptive

## **Environment & Configuration**

- Maintain separate environments: dev, staging, production
- API URLs and environment variables must be environment-specific
- Sensitive information must not be committed to Git

## **Dependency Management**

- Only approved packages may be used
- New package addition requires Tech Lead approval
- Packages must be actively maintained

## **Testing & Quality Checks**

### Mandatory

- Manual testing of assigned features and overall features
- Application must build successfully
- No runtime errors

## **Documentation Requirements**

Each project must include:

- Updated README.md
- Project setup instructions
- Environment configuration details
- Known limitations, if any

## **Code Review & Compliance**

- SOP compliance is mandatory for PR approval
- Reviewers must validate architecture, GetX usage, and code quality
- Repeated violations may impact performance evaluation